

SEC 160: FULL STACK DEVELOPMENT

COOKBOOK

Submitted by

K.TEJASRI	[AP23110011356]
CH.LOHITH SAGAR	[AP23110011348]
G.V.KARTHIKEYA	[AP23110011399]
K.SURYA	[AP23110011365]

SEM: V

SECTION: N

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

SCHOOL OF ENGINEERING AND SCIENCES



SRM University-AP, Neerukonda, Andhra Pradesh 522240

December 2025

INTRODUCTION

Cooking has always been an important part of human culture, but the way we store and share recipes has not changed much for many years. People moved from oral traditions to handwritten notes, then to printed cookbooks—but all of these methods are physical, easily damaged, and cannot change or update themselves.

Today, the digital age has changed many aspects of life, including how we cook. The **Cookbook** project is a major step in this digital shift. It provides a smart, web-based platform for managing recipes.

This project is more than just a digital place to store notes. **Cookbook** can understand the structure of a recipe—reading ingredients, identifying cooking times, and allowing users to interact with the data. Built using modern web technologies, especially the **React** ecosystem, the app is fast, responsive, and easy to use.

It solves the modern cook's need for quick access, organization, and clarity. Instead of messy bookmarks or scattered handwritten notes, users get a clean, searchable, and interactive recipe database. In this way, **Cookbook** connects the creative world of cooking with the structured, efficient world of software development.

Scenario-Based Intro: (The "Chaotic Dinner Party")

Sarah is a 34-year-old marketing manager who is hosting a dinner for 8 friends. She decides to cook *Chicken Tikka Masala*, but the recipe she finds only serves 4 people. Since she's not good at mental math, doubling amounts like **1.5 tablespoons** becomes confusing for her. On top of that, her hands are covered in marinade, so she can't keep unlocking her phone to check calculations.

With the cookbook app, she opens the recipe on her tablet and taps the **"Scale"** button. She selects a custom scaling factor, and the app instantly updates every ingredient for 8 servings. This saves her from doing any manual calculations.

Because her kitchen has dim lighting, she turns on **Dark Mode** so the screen is easier to read. When it's time to simmer the gravy for 20 minutes, she uses the built-in **Cooking Timer**, so she doesn't need another device or tool.

Thanks to these features, Sarah cooks easily without stress, avoids mistakes in measurements, and enjoys her evening with friends.

Target Audience:-

The Cookbook application serves a wide range of users, each with unique motivations and cooking habits. By understanding these groups, the system is designed to offer features that suit both everyday use and professional needs.

1. Home Cooks & Families

Home cooks form the largest segment of users. These are individuals or families who prepare meals regularly and require a convenient system to keep their recipes organized.

- **Needs:**
 - A simple way to store and categorize recipes.
 - Tools for meal planning throughout the week.
 - Ability to scale recipes based on the number of people—especially helpful for small families, large families, or when hosting guests.
- **Usage:**
 - Frequent daily use, often during evening meal preparation.
 - Quick access on tablets or phones while cooking.
 - Saving new family-favorite recipes or modifying existing ones for personal taste.

2. Professional Chefs & Caterers

These users work in high-pressure environments where precision and speed are essential. They need reliable access to recipes and instructions that can be scaled for larger events or batches.

- **Needs:**
 - Highly accurate ingredient measurements and cooking steps.
 - Fast access to standard operating procedures (SOPs) used in commercial kitchens.
 - Recipe scaling for large groups, events, buffets, or catering orders.
- **Usage:**
 - Access during prep time in commercial kitchens.
 - Reference for training new staff.
 - Maintaining consistency in taste and presentation across multiple cooks.

3. Fitness Enthusiasts

This group includes individuals focused on healthy eating, weight loss, muscle gain, or specific diet plans. They rely on recipes that meet their nutritional goals.

- **Needs:**
 - Ability to track ingredients and avoid unhealthy alternatives.
 - Filters for specific diets like Keto, Vegan, Gluten-Free, High-Protein, or Low-Carb.
 - Quick access to nutritional information or healthier substitutes.

- **Usage:**
 - Searching based on ingredients they have at home.
 - Filtering recipes by diet tags or calorie preferences.
 - Saving customized versions of recipes to match their fitness goals.

4. Culinary Students

Culinary students are learning the art and science of cooking. They need a tool that helps them study, practice, and build confidence as they progress.

- **Needs:**
 - A structured repository of recipes they can edit, annotate, or improve.
 - A space to store techniques, variations, and personal notes.
 - A way to build a professional portfolio they can showcase in the future.
- **Usage:**
 - As a study aid for understanding new cooking techniques.
 - Practicing and updating recipes as they learn.
 - Preparing for practical exams or building a recipe collection for internships.

Project Goals and Objectives

The development of the Cookbook application was driven by a set of clear strategic goals and practical technical objectives. These ensured that the platform not only functioned effectively but also delivered long-term value to its users.

Strategic Goals:

1. Centralization

Traditional recipe storage is scattered—handwritten notes, bookmarked web pages, screenshots, and multiple apps.

The goal of this project is to bring all these fragmented sources together into one **centralized platform**, creating a single “Source of Truth” for all recipes. This ensures that users no longer waste time searching through multiple places to find what they need.

2. Accessibility

Culinary knowledge should not be restricted by device or location.

The application is designed to run on **any device with a web browser**, whether it's a laptop, phone, or tablet. This ensures users can cook, learn, or browse from anywhere—at home, at work, or even while traveling.

3. Preservation

Physical recipes degrade over time due to fading ink, torn pages, or lost notebooks. A key goal of this project is to **preserve personal and family recipes digitally**, ensuring they remain safe, readable, and accessible for future generations.

Technical Objectives:

1. High Performance

The application aims to deliver a fast and smooth user experience.

- Goal: The app should load in **under 2 seconds**.
- Implementation: Achieved using **Vite's optimized bundling system**, which reduces build size and improves loading speed.

2. Responsive Design

Users access the app from various devices, so the interface must adjust gracefully.

- Goal: Provide a seamless UI on **Desktop (1920px), Tablet (768px), and Mobile (375px)**.
- Implementation: Flexible layouts, media queries, and reusable responsive components ensure a consistent experience across devices.

3. Data Integrity

Recipe data must be accurate, consistent, and reliable.

- Goal: Ensure all **CRUD operations** (Create, Read, Update, Delete) work without failure.
- Execution:
 - Deleted recipes should be permanently removed.
 - Updated recipes should show changes instantly.
 - No data duplication or corrupted entries.

4. Security

User data and recipes must be protected.

- Goal: Only authenticated users should have permission to modify recipes.
- Implementation:
 - Secure route guards with **ProtectedRoute**.
 - Restricted access ensures unauthorized users cannot change or delete data.

Key Features

The Cookbook application offers a wide range of features designed to enhance usability, performance, and user experience. Below is a detailed breakdown of each core functionality:

1. User Authentication System:

A secure login and authentication flow ensures that only authorized users can access or modify recipes.

- **Secure Login Page:** Includes input validation to prevent invalid or empty entries.
- **Session Persistence:** Uses **LocalStorage** to keep users logged in even after refreshing or closing the browser tab.
- **Automatic Redirect:** Once the user logs in, they are immediately taken to the main dashboard for quick access.

2. Recipe Gallery & Dashboard:

The central hub where users browse all recipes in a clean, visually appealing layout.

- **Grid-Based Recipe Cards:** Shows images, names, and basic recipe information.
- **Lazy Loading:** Images load only when needed, improving performance.
- **Quick Metadata View:** Tags for rating, preparation time, and difficulty help users make quick decisions.

3. Smart Search and Filtering Engine:

Designed to help users find the right recipe instantly.

- **Real-Time Search:** Instantly scans recipe titles and descriptions as the user types.
- **Advanced Filters:** Users can filter recipes by:
 - **Cuisine:** Indian, Italian, American
 - **Difficulty:** Easy, Medium, Hard
 - **Time:** Short, moderate, or long cooking durations
- **Multi-Select Support:** Users can apply multiple filters at once for precise results.

4. Recipe Management (CRUD Operations):

Complete control over recipe creation and editing.

- **Create:** Multi-step form for adding:
 - Title
 - Description
 - Ingredients (with an editable dynamic list)
 - Step-by-step instructions
 - Image URLs
- **Read:** Detailed recipe pages showing all information in a clean, readable format.

- **Update:** Edit existing recipes to fix mistakes or adjust ingredient quantities.
- **Delete:** One-click deletion for removing recipes permanently.

5. Dynamic Ingredient Scaling:

One of the most practical features that helps adjust recipes to match serving sizes.

- **Smart Algorithm:** Automatically detects numbers inside ingredient text (e.g., "1.5 tbsp", "2 cups").
- **Scaling Controls:** Simple + and – buttons to increase or decrease servings.
- **Instant Recalculation:** Ingredient quantities update immediately without page reload.

6. PDF Export Module:

Enables users to save or print their recipes for offline use.

- **Client-Side PDF Generation:** No server required—PDFs are created directly in the browser.
- **Custom Layout:** Includes styled headers, formatted text, ingredient lists, and steps.
- **Easy Download:** One click triggers the PDF download.

7. Favorites System:

Helps users build a personalized recipe collection.

- **Heart Icon Toggle:** Save or unsave recipes instantly.
- **Dedicated Favorites Page:** Quickly access all saved recipes in one place.
- **Persistent Storage:** Favorites are linked to the user's profile for long-term access.

8. Dark Mode / Theming:

A smooth visual experience that adapts to user preference or environment.

- **Theme Toggle:** Switch between Light Mode and Dark Mode anytime.
- **Modern Styling:** Uses CSS variables and Tailwind's `dark:` classes for instant theme changes.
- **Better Accessibility:** Ideal for late-night cooking or dimly lit kitchens.

PRE-REQUISITES

To install, run, or further develop the Cookbook application, certain hardware, software, and user knowledge requirements must be met. These ensure smooth performance and compatibility across devices.

1. Hardware Validation:

The system runs efficiently on most modern computers, but the following specifications are recommended:

- **Processor:**
 - Minimum: *Intel Core i3* or equivalent
 - Recommended: *Intel Core i5* or higher for a smoother development experience
- **RAM:**
 - Minimum: *4GB*, suitable for basic usage
 - Recommended: *8GB* to comfortably run development tools, Node servers, and the browser simultaneously
- **Storage:**
 - At least *500MB* of free disk space to store project files, node_modules, and build outputs

2. Software Environment:

Certain tools and versions are required to ensure the project builds and runs without compatibility issues:

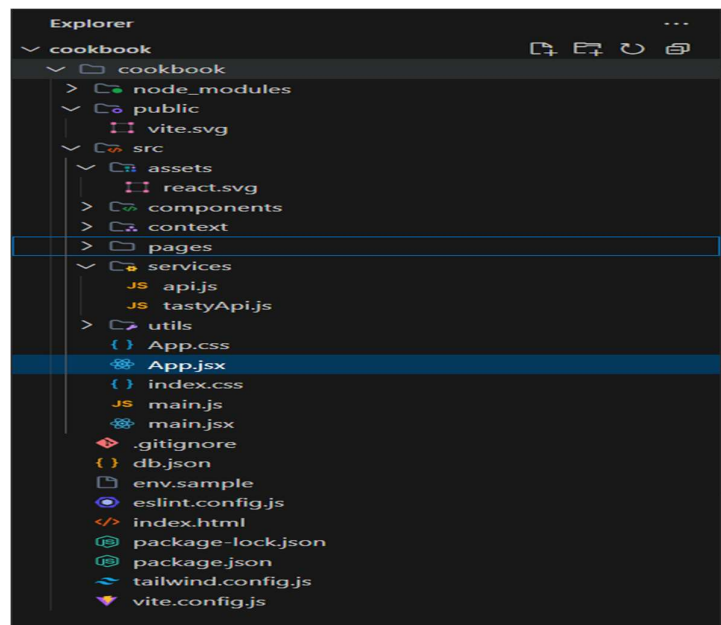
- **Node.js (v16.0.0 or higher):**
Needed to execute JavaScript outside the browser, run the development server, and build the React application.
- **NPM (v8.0.0 or higher):**
Package manager required for installing dependencies such as **React**, **Vite**, **Tailwind CSS**, and other libraries.
- **Modern Web Browser:**
Compatible Browsers:
 - Google Chrome (v90+)
 - Mozilla Firefox (v88+)
 - Microsoft EdgeThese support modern ES6+ features used in the project.
Not Supported: Internet Explorer, due to outdated JavaScript support.
- **Code Editor (for developers):**
 - **Visual Studio Code** (recommended)
 - Suggested extension: *ES7+ React/Redux/React-Native snippets* for improved coding efficiency

3. User Knowledge Requirements:

The application is designed to be accessible to both technical and non-technical users:

- **End Users:**
 - Only require basic knowledge of using a web browser.
 - No programming or technical skills are needed.
- **Developers (optional):**
 - Understanding of JavaScript, React, and Node.js is helpful for making changes or adding features.

Project structure:



This project follows a clean React + Vite folder structure. The **src** folder contains main working files like components, pages, context, services, and utilities. The **components** folder stores reusable UI parts, while **pages** contains full screens of the app. The **services** folder handles API calls, and **context** manages global state. Configuration files like **vite.config.js**, **package.json**, and styling files are kept at the root for easy setup and build management.

PROJECT FLOW:-

Project demo:

Before starting to work on this project, let's see the demo.

Demo link:

<https://drive.google.com/drive/folders/14qyLjiEnN8AfgnWS1JU0UJdKN0dww3eG?usp=sharing>

Use the code in:

https://drive.google.com/drive/folders/1cHVZoiaqB1ZlVSm4Bz8Q17xGipc_2vIZ?usp=sharing

Milestone 1: Project Setup and Configuration:

1. Initialization:

- We utilized *npm create vite@latest* to scaffold a bare-bones React application. Vite was chosen over Create-React-App for its superior build speed.

2. Dependency Installation:

- installed *react-router-dom* for handling multi-page navigation.
- installed *axios* for making HTTP requests to our backend.
- installed *react-icons* for accessing FontAwesome and Material Design icons.

3. Tailwind CSS Integration:

- Ran *npm tailwindcss init -p* to generate config files.
- Configured *content* array in *tailwind.config.js* to scan our JSX files.
- Added *@tailwind* directives to our *index.css*

4. Backend Simulation:

- Set up *json-server*.
- Created *{ db.json}* and populated it with dummy data (3 initial recipes with diverse tags) to test data retrieval.

5. Linting & Quality Control:

- Set up ESLint rules to catch potential React hooks errors and unused variables.

Milestone 2: Web Development:

1.Setup React Application

The setup process involves initializing the project structure and installing necessary dependencies. We use Vite for its superior performance over Create-React-App.

Step-by-Step Configuration:

Project Initialization:

```
npm create vite@latest cookbook -- --template react
cd cookbook
```

Dependency Installation: We install the core libraries required for routing, state management, and styling.

```
npm install
npm install react-router-dom axios react-icons react-toastify jspdf
```

Tailwind CSS Setup: To enable utility-first styling:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Backend Simulation: We use json-server to mock a REST API.

```
npm install -D json-server
```

Created {} db.json in the root directory.

2.Design UI Components:

We adopted the Atomic Design capability, breaking down the interface into reusable, modular components.

Component Breakdown:

Atoms (Basic Elements):

- Button: Styled with Tailwind classes bg-amber-600 hover:bg-amber-700 text-white.
- Input: Standardized text fields with focus rings.

Molecules (Simple Groups):

- **Rating.jsx** : A component that takes a score prop and renders dynamic star icons (FaStar).
- **SearchBar.jsx** : Combines an input field with a search icon and a clear button.

Organisms (Complex Structures):

- **RecipeCard.jsx** : The primary display unit. It composes the Image, Title, Rating molecule, and Action buttons (View, Delete) into a cohesive card.

- **Navbar.jsx** : The sticky header containing the Logo, Navigation Links, and Theme Toggle.

Design System:

- **Color Palette:** Slate (900/800) for dark backgrounds, Amber (500/600) for primary actions.
- **Typography:** Sans-serif (Inter/System UI) for readability.
- **Effects:** Glassmorphism (backdrop-blur-md) used on the Navbar and Overlay cards.

3. Implement Frontend Logic

The core logic is distributed between Context Providers for global data and Custom Hooks for component logic.

Key Logic Implementations:

Routing Logic (App.jsx):

- Uses Routes and Route to map URLs to components.
- **Route Guard:** The ProtectedRoute component checks isAuthenticated. If false, it uses `<Navigate to="/login" />` to redirect users, securing the application.

Global State (AuthContext.jsx):

- **Problem:** User login state is needed by the Navbar (to show user name) and ProtectedRoute (to allow access).
- **Solution:** created a Context that provides user, login(), and logout() functions to the entire component tree. It persists state using localStorage.setItem.

Data Fetching (api.js):

- Abstracted all fetch/axios calls into a service layer.
- Example: `api.get('/recipes')` is called inside a useEffect hook in Home.jsx.
- Loading States: While data fetches, a loading state variable displays a spinner.

Feature Logic (Scaling):

- Located in RecipeScaling.jsx.
- Logic: $\text{NewAmount} = (\text{BaseAmount} / \text{BaseServings}) * \text{TargetServings}$.
- Regex parsing separates units ("cups") from quantities ("2.5") to perform math on just the number.

4. Code-Editor Component (RecipeForm.jsx):

Note: In the context of this Cookbook application, the "Code-Editor" refers to the complex Recipe Creation/Editing Form, which allows users to "program" the recipe structure.

Component Overview

The RecipeForm.jsx is the most complex component in the application, handling data entry for structured content.

Functionality:

1.Dynamic Input Arrays:

- Recipes have lists of ingredients and instructions. Users can add or remove items dynamically.

Logic:

```
const addIngredient = () => {  
  setFormData({ ...formData, ingredients: [...formData.ingredients,  
    '' ] });  
};
```

Two-Way Binding:

- Every input field is bound to the formData state object. Changes update the state instantly.

Mode Detection:

- The component supports both Create and Edit modes.
- It checks the URL for an ID (/recipes/edit/:id). If an ID exists, it fetches the existing data and pre-fills the form.

Validation:

- Prevents submission if crucial fields (Title, Ingredients) are empty.
- Provides visual feedback (toast notifications) on success or error.

Code Structure Snippet:

```

// RecipeForm.jsx Logic
const handleSubmit = async (e) => {
  e.preventDefault();
  // 1. Validate Data
  if (!formData.title) return toast.error("Title required");

  // 2. Send to API
  if (id) {
    await api.updateRecipe(id, formData); // Edit Mode
  } else {
    await api.createRecipe(formData); // Create Mode
  }

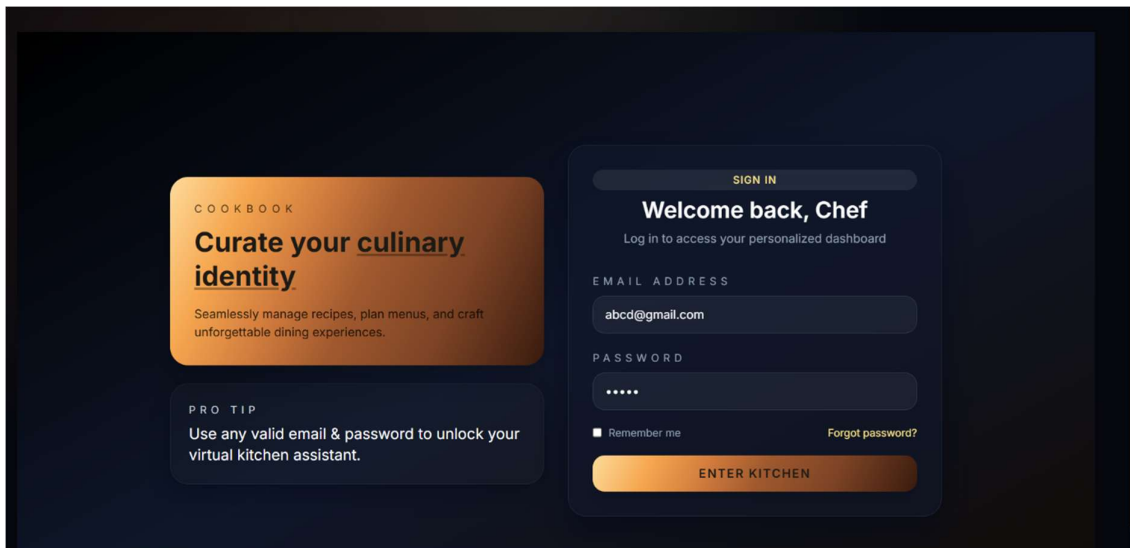
  // 3. Redirect
  navigate('/');
};

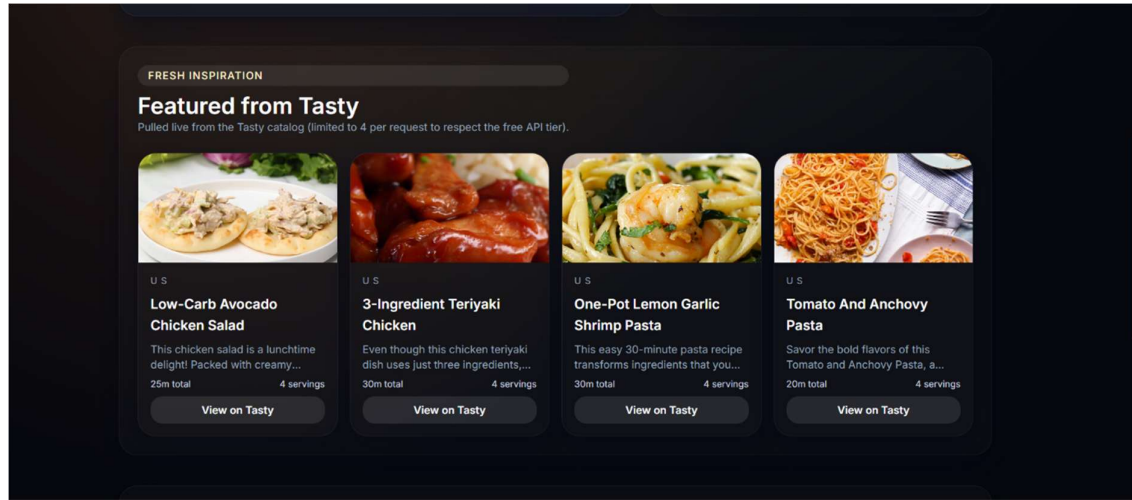
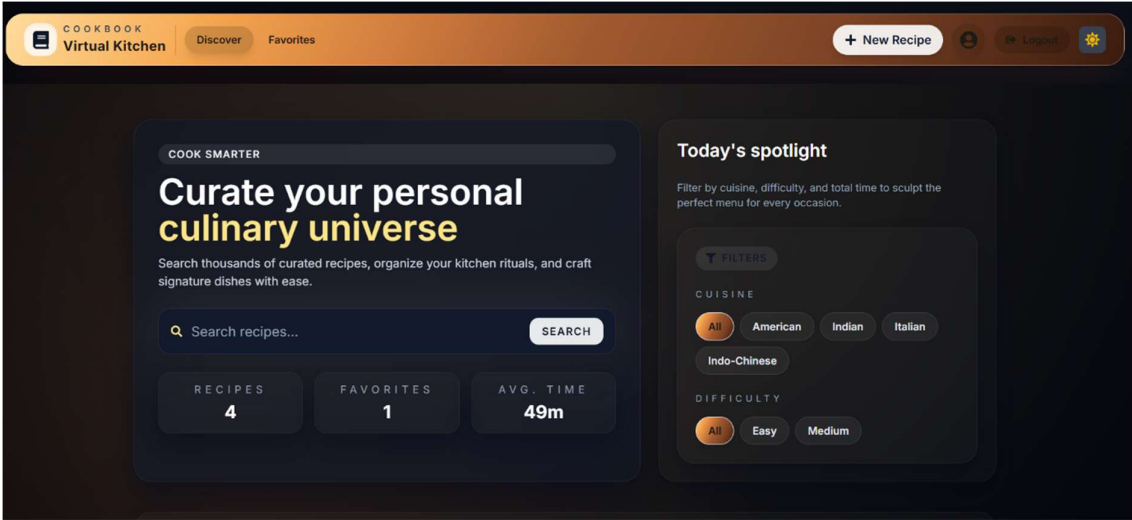
```

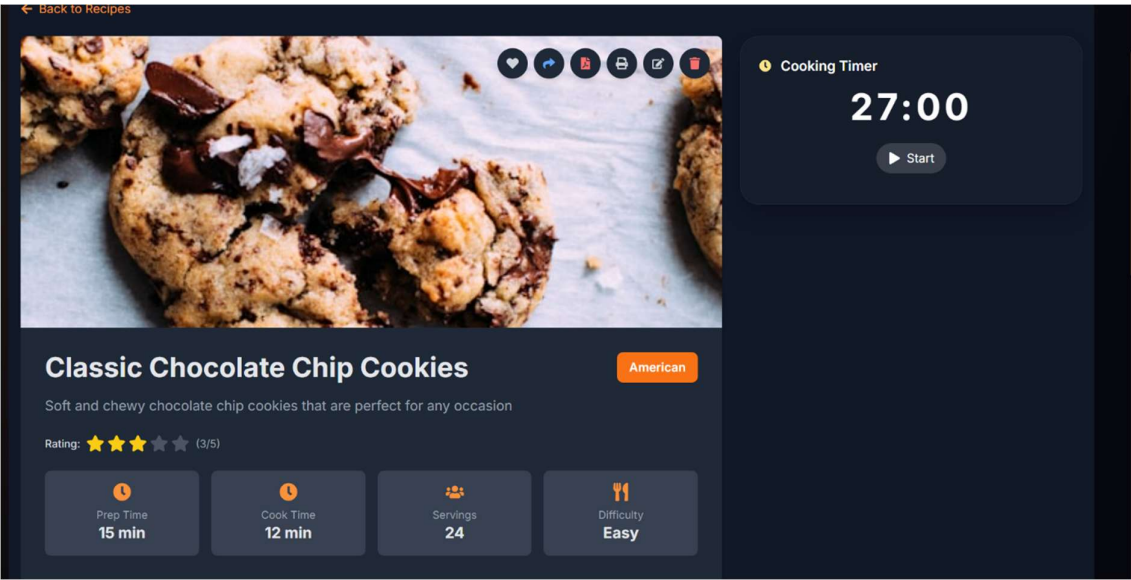
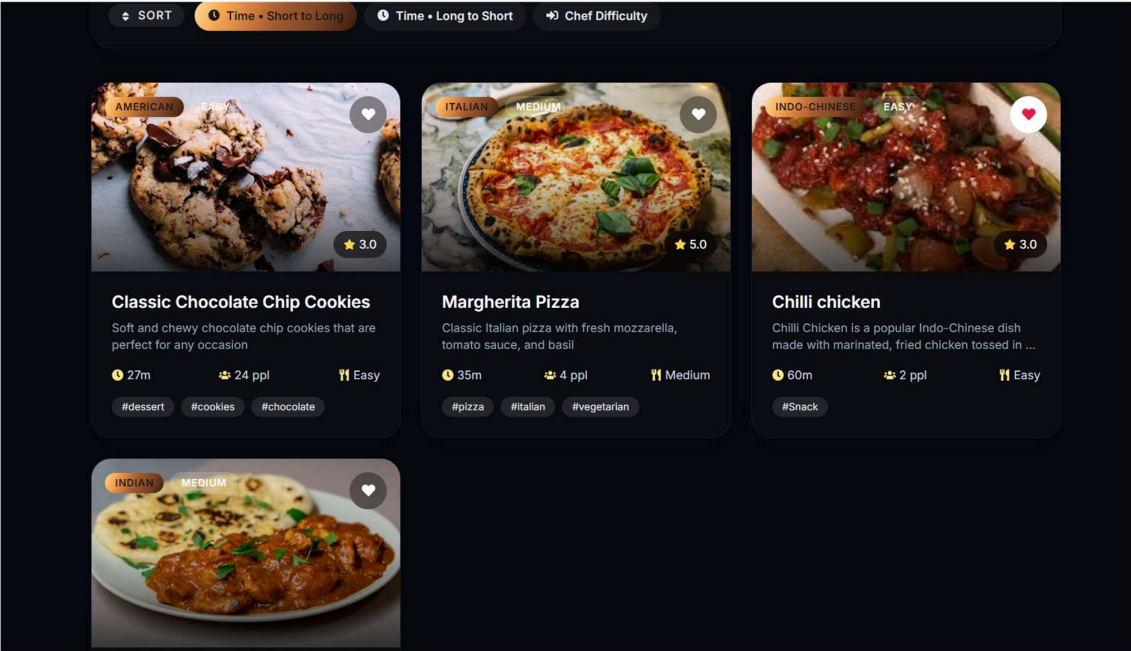
Project Execution:

After completing the code, run the react application by using the command “npm start” or “npm run dev” if you are using vite.js

Here are some of the screenshots of the application.







Tags

dessert

cookies

chocolate

SCALE RECIPE

Servings control

−

24 servings

+

SCALED INGREDIENTS

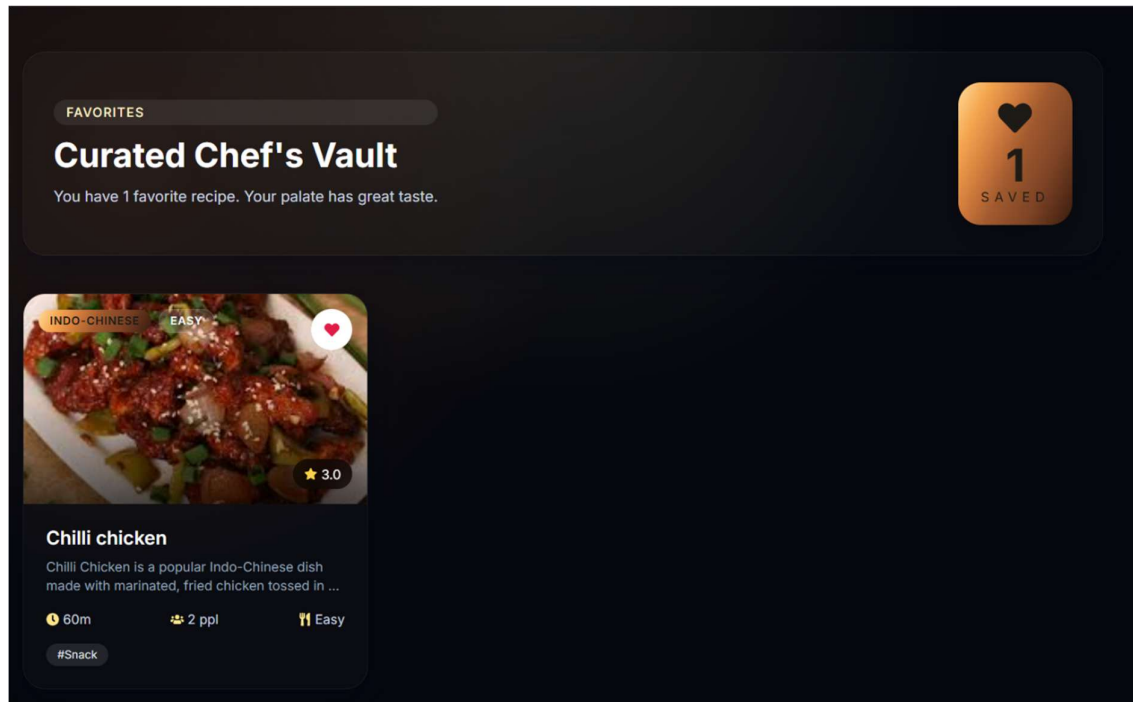
- 2.00 1/4 cups all-purpose flour
- 1.00 tsp baking soda
- 1.00 tsp salt
- 1.00 cup butter, softened
- 3.00 1/4 cup granulated sugar
- 3.00 1/4 cup brown sugar
- 2.00 large eggs
- 2.00 tsp vanilla extract
- 2.00 cups chocolate chips

Ingredients

- 2 1/4 cups all-purpose flour
- 1 tsp baking soda
- 1 tsp salt
- 1 cup butter, softened
- 3/4 cup granulated sugar
- 3/4 cup brown sugar
- 2 large eggs

Instructions

- 1 Preheat oven to 375°F (190°C)
- 2 Mix flour, baking soda, and salt in a bowl
- 3 Cream butter and sugars until smooth
- 4 Beat in eggs and vanilla
- 5 Gradually blend in flour mixture



Github link:

<https://github.com/tejukatlagunta91/Cookbook>